



OO REVIEW CSC 275



Terminology

- ▣ Class
- ▣ Object
- ▣ Encapsulation
- ▣ Constructor
- ▣ Overloading



Class



- ▣ Structure definition for an entity, the information to be stored about the entity, and the operations that can be performed on it.



Class



- ▣ Structure which defines an entity
- ▣ 3 parts
 - Name
 - Attributes
 - Methods



Class Example

▣ Name →

▣ Attributes →

▣ Methods →

Employee	
	Name
	Address
	SSN
	Start Date
	Wage Rate
	Employee()
	Hire()
	Promote()



Why use Classes?

Employee



Name



Address



SSN



Start Date



Wage Rate



Employee()



Fire()



Promote()

- ▣ Operational Integrity
- ▣ Reusability
 - HR
 - Payroll
 - Management
 - Contracts
- ▣ Expandability
 - Part-time
 - Full-time
- ▣ Consistency
- ▣ Data Integrity



Object



- ▣ Instance of a class



Encapsulation

- ▣ Hiding of implementation details and Data integrity
 - Ex. Fire Employee Method
 - ▣ Set termination date = TODAY
 - ▣ Notify HR, Payroll, Security, MIS
 - Ex. Promote
 - ▣ Checks raise amount must be $< 10\%$ or manager override
 - ▣ Cannot be retroactive > 3 months



Encapsulation Purpose



- ❑ Insure attribute manipulation is done properly.
- ❑ Doesn't allow access except through methods so control is invoked.
- ❑ Allows consistency so each time the same series of events/checks/changes occur.



Protection

▣ Protection levels are used to enforce encapsulation

▣ Levels:

Security



- Public: Open access to all
- Protected: Class itself and inherited classes have access
- Private: Only the class itself has access



Protection Assignments



- ▣ USUALLY,
- ▣ Attributes are PRIVATE
 - Sometimes protected in inherited classes
- ▣ USUALLY Methods are PUBLIC



Constructors

- ▣ Special method used to create a new instance of a class.
 - In the implementation, constructors must have the same name as the class. In the A & D portion, they may contain a more descriptive name.
 - Ex. Class Employee = Employee()
- ▣ Every class must have a constructor



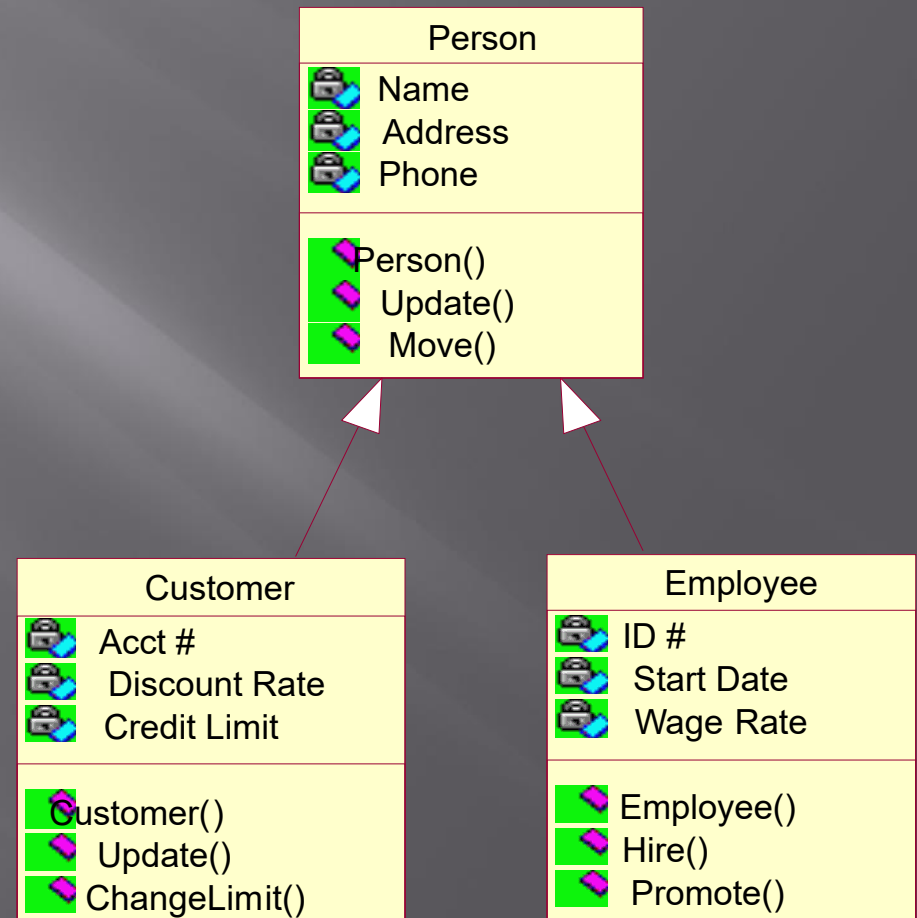
Inheritance

- OO concept where subclass inherits all attributes and methods from a more general superclass.
- Also known as generalization/ specialization because superclass is generic, subclasses are more specific



Inheritance Example

- Customer & Employee get all attributes from parent class Person
- Customer & Employee get all methods from Person, but can be overridden by writing a new method in child classes

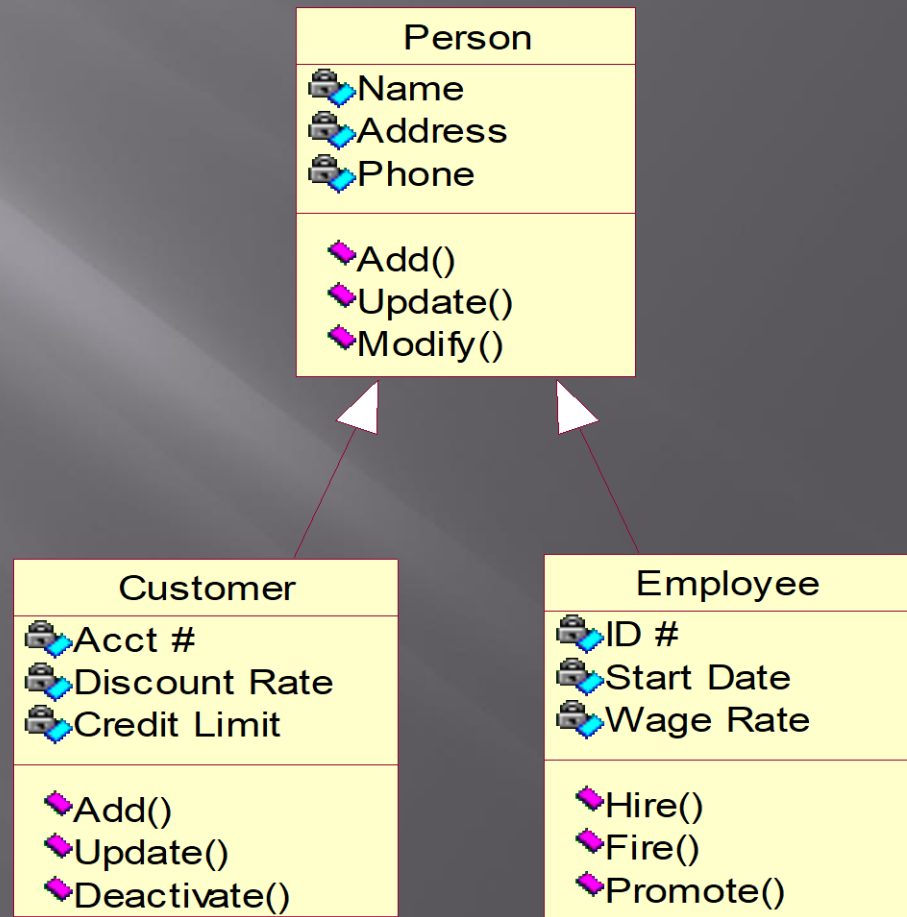




Abstract Class

- When there are no instances (objects) of a class, it is called an **ABSTRACT CLASS**

- In this Example, Person is an abstract class





More on Inheritance

- ▣ Purpose
 - Reuse
 - Maintainability
 - Consistency
- ▣ Must have at least 2 subclasses
 - Complication
 - otherwise, no reuse is possible



Yet more on Inheritance



- ▣ Must be able to say:
 - “Subclass is a type of superclass”
- ▣ MUST use all attributes
- ▣ Subclasses can add additional attributes, methods
- ▣ Subclasses can override inherited methods

